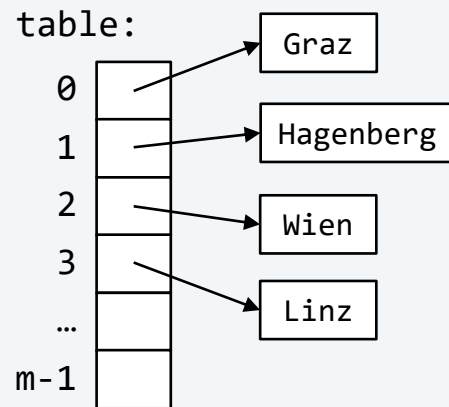


7 Hashing-basierte Suche



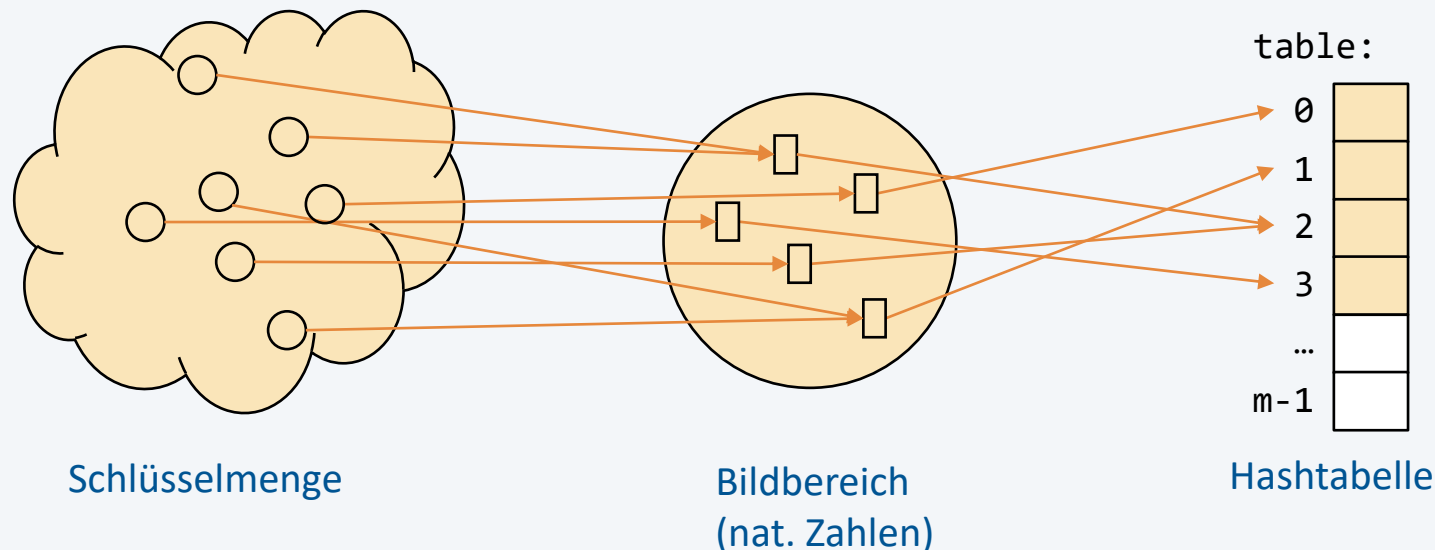
7.1 Kollisionsbehandlung: Verkettung

7.2 Kollisionsbehandlung: Offene Adr.

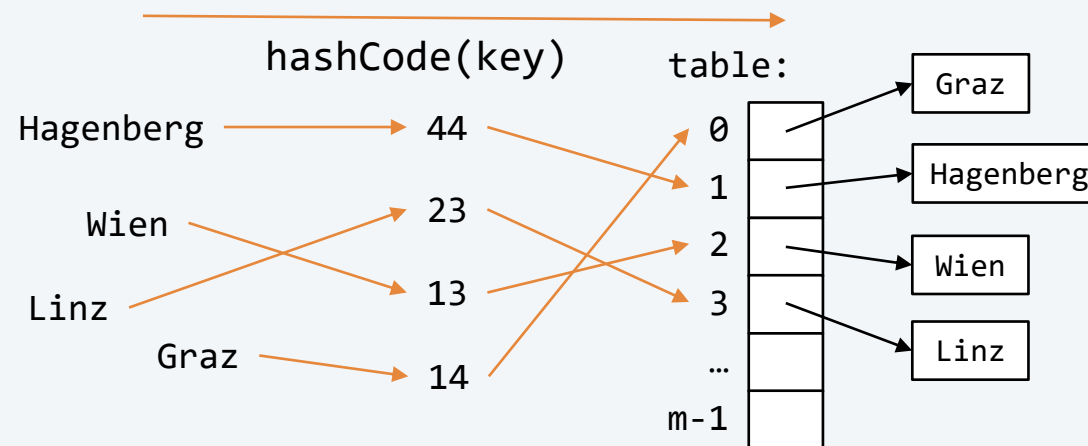
7.3 Wahl der Hashfunktion

- bei den bisher erörterten Suchverfahren, der sequenziellen und der binären Suche, hängt der **Aufwand** für die Lokalisierung eines gesuchten Elements von der **Anzahl n der Elemente** im Behälter ab
 - die asymptotische Laufzeitkomplexität von sequenziellen Suchverfahren ist $O(n)$
 - die asymptotische Laufzeitkomplexität binärer Suchverfahren ist günstiger, sie ist nur $O(\log n)$
- obwohl es sich bei binären Suchverfahren bereits um effiziente Verfahren handelt, ist es wünschenswert und in bestimmten Fällen erforderlich, **noch effizientere Suchverfahren** zur Verfügung zu haben
- wünschenswert wäre ein Suchverfahren, das **unabhängig von der Anzahl** der Elemente im Behälter mit einer asymptotischen Laufzeitkomplexität von annähernd $O(1)$ arbeitet, dessen Laufzeitverhalten also konstant ist

- das sogenannte **Hashing**, ist ein Verfahren, mit dem die Forderung einer asymptotischen Laufzeitkomplexität von $O(1)$ – unter bestimmten, aber **realistischen Umständen** – **weitgehend** erfüllt werden kann
- Hashing basiert auf der Idee, dass man bereits beim Speichern von Datenobjekten in einem Behälter dafür Sorge trägt, dass die einzelnen Datenobjekte in Abhängigkeit von einem Suchschlüssel schnell, möglichst auf Anhieb, gefunden werden können



- Verwendung eines Felds fixer Größe m
- Transformation des Schlüsselinhalts in eine natürliche Zahl
- Abbildung der Zahl auf das Index-Intervall $[0..m-1]$
- Speichern der Datenobjekte, sodass Zugriff über berechn. Feldindex möglich ist



- Hashing bedeutet also die Abbildung von Schlüsselwerten auf einen Wert (Index) aus dem Intervall $[0..m-1]$
- die Funktion dazu heißt **Hashfunktion** `int hashCode(key)`, ihr Ergebnis nennen wir **Hashcode**

- gegeben seien Datenobjekte in Form von Knoten mit einer Schlüsselkomponente vom Datentyp String

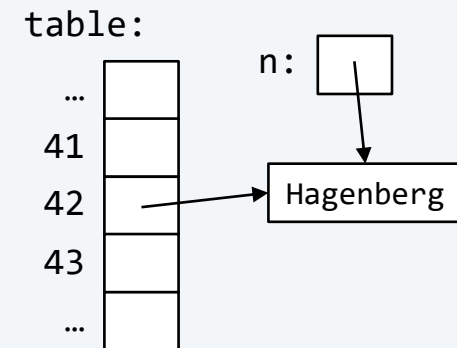
```
class Node {  
    String key;  
    ... data;  
}
```

- gebildet wird eine Hashtabelle, d. h. ein Feld fixer Größe m

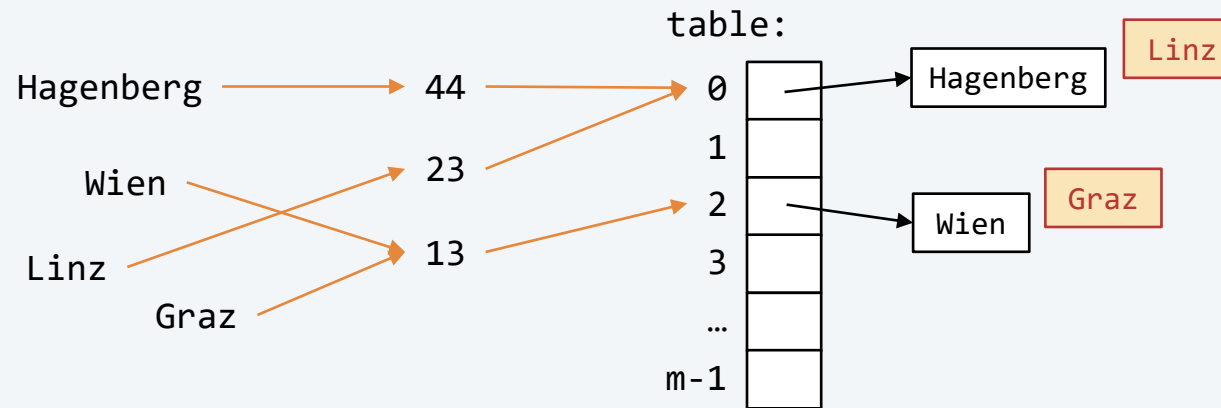
```
int m = ...;  
Node[] table = new Node[m];
```

- die Elemente repräsentieren Zeiger auf Knoten

```
Node n = new Node();  
n.key = "Hagenberg";  
table[42] = n;
```

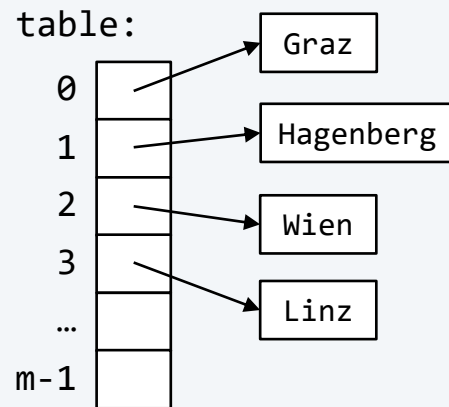


- Wenn für Schlüsselwerte $k_1 \neq k_2$ die Hashfunktion H identische Ergebnisse liefert, $H(k_1) = H(k_2)$, nennen wir dies eine Kollision
- Verschiedene Schlüsselwerte können auf die gleiche Zahl und/oder verschiedene Zahlen können auf den gleichen Index abgebildet werden



- Lösung des Kollisionsproblems
 - durch Verkettung von Kollisionsobjekten
 - durch offene Adressierung

7 Hashing-basierte Suche

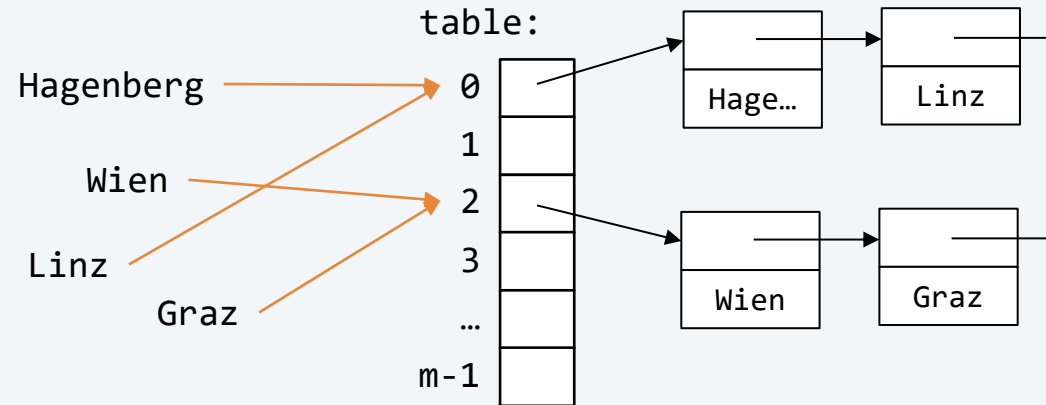


7.1 Kollisionsbehandlung: Verkettung

7.2 Kollisionsbehandlung: Offene Adr.

7.3 Wahl der Hashfunktion

Datenobjekte mit gleichem Hashcode werden in einer einfach-verketteten Liste gespeichert



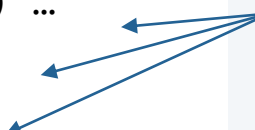
Zusätzliche Komponente next erforderlich

```
class Node {  
    String key;  
    String data;  
    Node next;  
  
    Node(String key, String data) {  
        this.key = key;  
        this.data = data;  
        this.next = null;  
    }  
}
```


Klasse Hashtable

```
public class Hashtable {  
    private Node[] table;  
    private int size;  
  
    public      Hashtable(int capacity) ...  
    public void put(String key, String data) ...  
    public String get(String key) ...  
    public void remove(String key) ...  
    public int size() { return size; }  
    private int hashCode(String key) ...  
}
```

Methoden auf den
nächsten Seiten



Initialisierung (Konstruktor)

```
public Hashtable(int capacity) {  
    assert capacity >= 0;  
    this.table = new Node[capacity];  
    this.size = 0;  
}
```

Methode put

```
public void put(String key, String data) {  
    assert key != null;  
    int i = hashCode(key);  
  
    if (table[i] == null) {  
        table[i] = new Node(key, data);  
        size++;  
    } else {  
        Node n = table[i];  
        while (!n.key.equals(key) && n.next != null) {  
            n = n.next;  
        }  
        if (!n.key.equals(key)) {  
            n.next = new Node(key, data);  
            size++;  
        }  
    }  
}
```

noch kein Eintrag mit
HashCode i

suche ob Knoten für
Schlüssel key bereits
vorhanden

noch kein Eintrag eines
Datenobjektes mit
Schlüssel key

Methode get

```
public String get(String key) {  
    assert key != null;  
    Node n = table[hashCode(key)];  
    while (n != null && !n.key.equals(key)) {  
        n = n.next;  
    }  
    if (n != null) {  
        return n.data;  
    } else {  
        return null;  
    }  
}
```



suche Knoten mit
Schlüsselwert key

Optimierungsmöglichkeit

- Datenobjekte sortiert nach Schlüsselwerten in Listen einfügen
- Suche kann abgebrochen werden, wenn aktueller Schlüsselwert größer als gesuchter Schlüsselwert ist

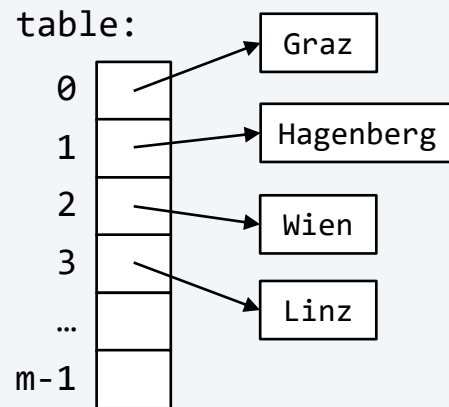
Methode remove

```
public void remove(String key) {  
    assert key != null;  
    int i = hashCode(key);  
    Node n = table[i];  
    Node prev = null;  prev zeigt auf Vorgänger des  
                        Knoten mit Schlüssel  
    while (n != null && !n.key.equals(key)) {  
        prev = n;  
        n = n.next;  
    }  
    if (n != null) {  
        if (prev == null) {  erster Knoten wird entfernt  
            table[i] = n.next;  
        } else {  innerer oder letzter  
                  Knoten wird entfernt  
            prev.next = n.next;  
        }  
        size--;  
    }  
}
```

- unter Annahme einer Gleichverteilung der Hashcodes ist die durchschnittliche Listenlänge $L = n/m$
 - mit n = Anzahl der Datenobjekte und m = Tabellengröße
- kennt man n , kann man m passend wählen



7 Hashing-basierte Suche



7.1 Kollisionsbehandlung: Verkettung

7.2 Kollisionsbehandlung: Offene Adr.

7.3 Wahl der Hashfunktion

- Datensätze auch bei Kollisionen in Tabelle speichern (für $n < m$)
- Verschiedene Strategien um "Ersatzposition" zu finden

Lineare Kollisionsstrategie

- Wenn Position belegt ist, wird erste freie Position dahinter gesucht

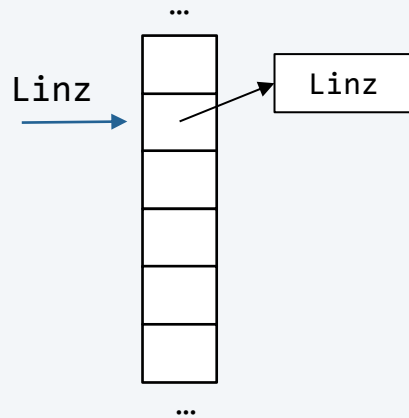
```
private int keyPos(String key) {  
    int i = hashCode(key);  
    while (table[i] != null && !table[i].key.equals(key)) {  
        i = (i + 1) % table.length;  
    }  
    return i;  
}
```

Hier ohne Prüfung, ob überhaupt
noch eine Position frei ist!

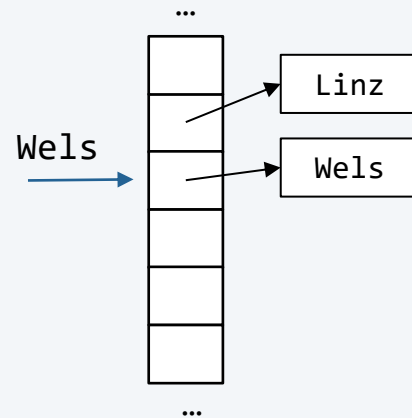
Methode put

```
public void put(String key, String data) {  
    assert key != null;  
    int i = keyPos(key);  
    table[i] = new Node(key, data);  
}
```

Beispiel

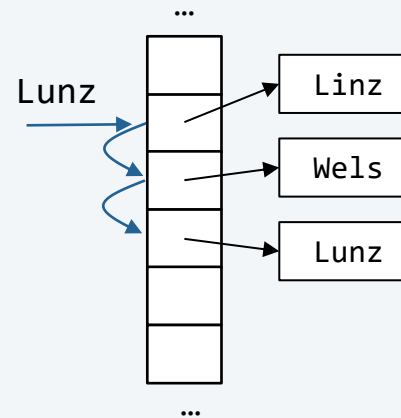


1. Insert Linz



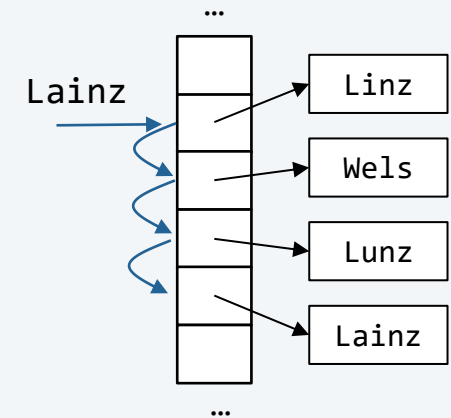
2. Insert Wels

$hc("Wels") \neq hc("Linz")$



3. Insert Lunz

$hc("Lunz") = hc("Linz")$



4. Insert Lainz

$hc("Lainz") = hc("Linz")$

Methode get

```
public String get(String key) {  
    assert key != null;  
    int i = keyPos(key);  
    if (table[i] != null) {  
        return table[i].data;  
    } else {  
        return null;  
    }  
}
```

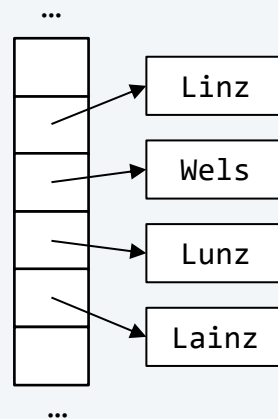
- bei dieser „linearen“ Platzierung und Suche ist nach Morris die mittlere Anzahl e von erforderlichen Versuchen das $n + 1$. Datenobjekt zu speichern oder zu finden $e = \frac{1 - \frac{a}{2}}{1 - a}$ mit $a = \frac{n}{m}$
- daraus folgt, dass die mittlere Anzahl von Suchschritten nicht von der Anzahl der Datenobjekte, sondern nur vom **Füllfaktor a** der Hashtabelle abhängt - der Füllfaktor lässt sich steuern

- man kann auch so vorgehen, dass bei einer Kollision nicht die nächste freie Zelle der Hashtabelle gewählt wird, sondern die Adresse mit einem Zufallszahlengenerator bestimmt wird
- ist auch diese Zelle belegt, wird der Vorgang so lange wiederholt, bis eine leere Zelle gefunden ist
- der Zufallszahlengenerator muss so konstruiert sein, dass er alle Zellen der Tabelle einmal berührt
- zum Suchen muss derselbe Generator verwendet werden, wie zum Speichern (gleicher Startwert)

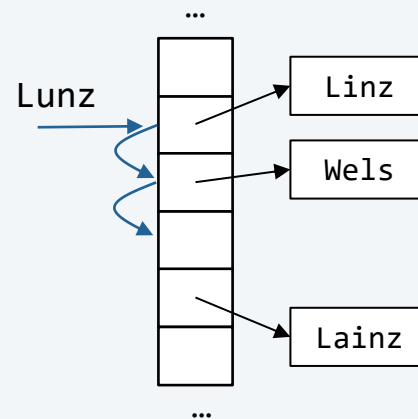
a	lineares Speichern	zufälliges Speichern
0.10	1.06	1.05
0.50	1.5	1.39
0.75	2.5	1.83
0.90	5.5	2.56

- nicht trivial
- Eintrag darf nicht auf null gesetzt werden, falls dahinter Datensätze mit gleichem Hashcode gespeichert sind

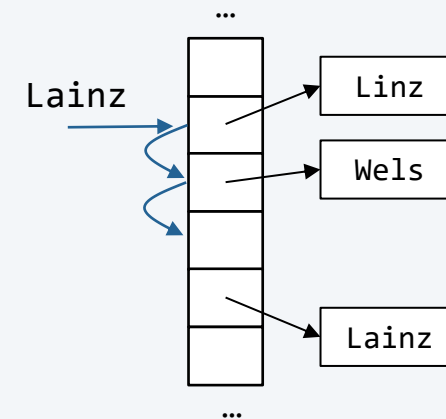
Beispiel



1.



2. Delete Lunz



3. Find Lainz

Lösungsmöglichkeiten

- spezieller Datensatz für gelöschte Elemente
- spezielles Flag für gelöschte Elemente

Spezielles Datenobjekt anstelle entfernter Datenobjekte

Deklaration

```
private final static Node NODE_DELETED = new Node("_♥@null@♥_", null);
```

Schlüsselwert darf sonst
nicht vorkommen

Methode remove

```
public void remove(String key) {  
    assert key != null;  
    int i = keyPos(key);  
    if (table[i] != null && table[i] != NODE_DELETED) {  
        table[i] = NODE_DELETED;  
    }  
}
```

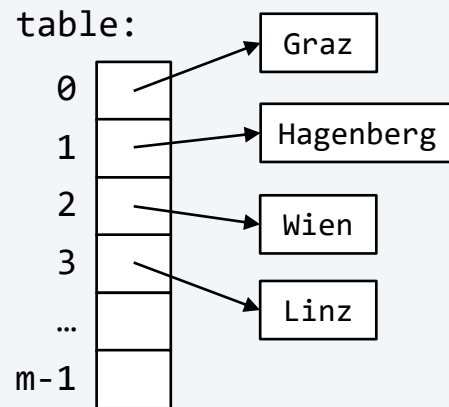
Erfordert Änderungen in den Methoden keyPos sowie in get

Änderungen in keyPos sowie in get

```
private int keyPos(String key) {
    int i = hashCode(key);
    int del = -1;
    while (table[i] != null && !table[i].key.equals(key)) {
        if (del < 0 && table[i] == NODE_DELETED) {
            del = i;
        }
        i = (i + 1) % m;
    }
    if (table[i] == null && del >= 0) {
        return del;
    } else {
        return i;
    }
}
```

```
public String find(String key) {
    Node n = table[keyPos(key)];
    if (n == NODE_DELETED) {
        return null;
    } else if (n != null) {
        return n.data;
    } else {
        return null;
    }
}
```

7 Hashing-basierte Suche



7.1 Kollisionsbehandlung: Verkettung

7.2 Kollisionsbehandlung: Offene Adr.

7.3 Wahl der Hashfunktion

Eine Hashfunktion ist dann gut, wenn der Hashcode **schnell** berechnet werden kann, möglichst **gleichverteilte** Hashcodes geliefert werden und daher möglichst wenig Kollisionen auftreten

Einfache Hashfunktionen für Zeichenketten als Schlüssel

Beispiel 1: Ordnungszahl des 1. Zeichens

```
int h = key.charAt(0);
```

- Besonders kollisionsanfällig

bauer	berger	böhm	raab
-------	--------	------	------

linz	lunz	langau	zwettl
------	------	--------	--------

- Schnell zu berechnen jedoch schlechte Verteilung

Beispiel 2: Ordnungszahlen des ersten und letzten Zeichens

```
int h = key.charAt(0) + key.charAt(key.length() - 1);
```

- Ebenfalls kollisionsanfällig

bauer berger böhm raab

linz lunz langau zwettl

Beispiel 3: Ordnungszahlen aller Zeichen

```
int h = 0;
for (int i = 0; i < key.length(); i++) {
    h = h + key.charAt(i);
}
```

- Kollisionen bei Permutationen, da Position der Zeichen nicht berücksichtigt wird

lampe = 108 + 97 + 109 + 112 + 101 = 527
palme = 112 + 97 + 108 + 109 + 101 = 527

Beispiel 4: Hashfunktion nach Rabin u. Karp: Ordnungszahlen aller Zeichen + Position

```
int h = 0;
for (int i = 0; i < key.length(); i++) {
    h = (h * d + key.charAt(i)) % m;
}
```

- mit d = Anzahl der möglichen Werte von `key.charAt(i)` (ASCII: $d = 256$) und m = gewünschter Wertebereich (Tabellengröße)
- die Größen d und m müssen teilerfremd sein
- liefert gleich verteilte Werte aus $0 \dots m - 1$ (unter der Bedingung, dass $\text{Gcd}(d, m) = 1$)

Abbildung auf das Indexintervall der Hashtabelle $[0, m-1]$

- Hashfunktion nach Rabin u. Karp liefert gleich passende Werte zwischen 0 und $m - 1$, die anderen Funktionsergebnisse müssen erst noch in diesen Bereich gebracht werden.

$$h = h \% m;$$

- Schlecht, da benachbarte Elemente benachbart bleiben

$$\begin{aligned} 2041 \% 1000 &= 41 \\ 2042 \% 1000 &= 42 \end{aligned}$$

- Kann also zu Häufungen führen, liefert nur gute Ergebnisse wenn m eine Primzahl ist
- Besser Multiplikation mit Konstante a wie bei linearer Kongruenzmethode

$$h = (a * h) \% m;$$

$$\begin{aligned} 2041 * 621 \% 16384 &= 5893 \\ 2042 * 621 \% 16384 &= 6514 \end{aligned}$$